

FinanceBench: Vector RAG vs. Vectorless RAG vs. Long Context

Master's thesis project walkthrough · Philipps-Universität Marburg · Supervisor: Prof. Dr. Daniel Braun · 29 September 2026

1. The question

Companies' SEC filings run to hundreds of pages, and answering a question about one with an LLM needs a way to get the right text in front of the model. I compared three established approaches and one of my own: **vector RAG** (search for the most similar chunks), **vectorless RAG** (let an LLM navigate the document's table of contents), a **PageIndex hybrid** I designed (navigate first, then search inside the chosen pages), and **long context** (send the whole filing). The goal: an accuracy–latency–cost Pareto frontier showing which approach to use when.

Dataset: FinanceBench open-source split: 150 questions over 84 real 10-K/10-Q filings, with gold answers and gold evidence pages. 50 questions each are *metrics-generated* (calculations), *domain-relevant* and *novel-generated* (analyst-style judgement questions).

2. What I built

Architecture	How it finds the evidence	What the model reads (median)
Vector RAG (Kim et al., 2025)	Query expansion → finance-tuned Stella embeddings + BM25 ($\alpha = 0.85$, top 20) → Jina rerank (top 10) → LLM selection agent	1–2 chunks, ~0.8k tokens
Vectorless RAG (PageIndex, Lumer et al., 2025)	Each filing turned once into a section tree (PageIndex Flash); per question, query expansion → LLM picks sections from the tree	~4 whole sections, ~33k tokens
PageIndex hybrid (my design)	PageIndex navigation restricts the search to the chosen pages → vector RAG's hybrid search + rerank inside them; no selection agent	10 chunks, ~4.4k tokens
Long context	No retrieval: the whole PDF is sent with every question, no caching	whole filing, ~70k tokens

Held constant for all four: the generator (*gemini-3.1-flash-lite*), one structured prompt (preserve units, show any calculation, answer as JSON), one scoring rule, and the judge (*gpt-oss-120b* via Groq, a different model family). Measured: accuracy (Correct / Incorrect / Failure to answer), retrieval quality, cost per query and end-to-end latency.

3. Building it: what went wrong and how I fixed it

- **Free-tier limits.** Gemini 3.5 Flash allowed only 20 requests a day, and Voyage rerank had a tight rate cap. I switched every pipeline to *gemini-3.1-flash-lite* and used Jina rerank instead.
- **PageIndex's tree builder failed on many filings.** Its classic builder needs the LLM to return a strictly formatted table of contents, which flash-lite often couldn't. I rebuilt all 84 trees with **PageIndex Flash**, which reads the structure from the PDF layout.
- **Trees silently missing whole parts of documents.** Flash found no headings before page 85 of 3M's 2018 10-K, so those pages were unreachable. I added a coverage check and wrote my own **LLM table-of-contents fallback**: one LLM call proposes the sections, code fills any gaps, and each section gets a summary. It was used for 16 documents plus a few flagged by a later audit.
- **Navigation missed sections when the question used everyday words** (“cash on hand”) instead of filing terms (“Cash and Cash Equivalents”). Query expansion before navigation raised the hit rate from 92.7% to 94.0%.
- **Resumable stages can't give an end-to-end latency.** Each stage ran on its own schedule, so I added a dedicated sequential timing pass per pipeline (10 questions × 3 repeats).
- **Cost logs contained duplicates.** Re-runs left up to 14 records per question, and indexing costs were undercounted. I de-duplicated by matching each record to the answer actually saved, fixed the indexing clean-up, and estimated the 36 trees built before cost logging worked.

4. Making the comparison fair

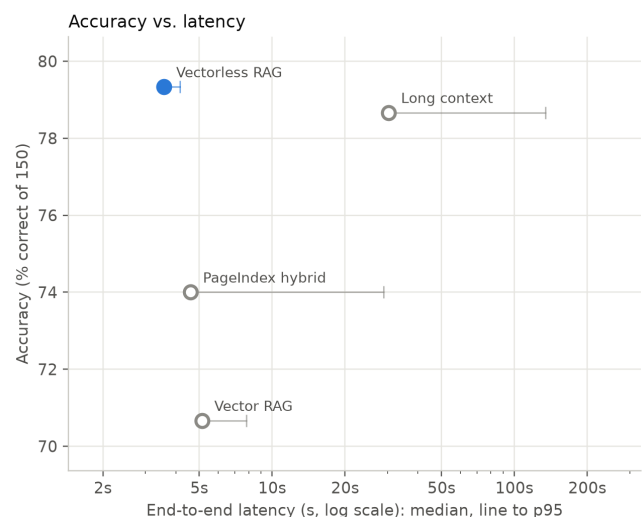
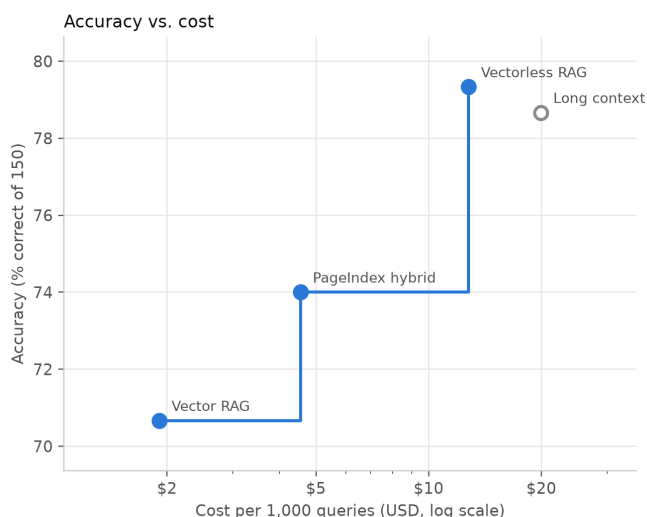
Two inconsistencies surfaced during the analysis, and each changed the results noticeably:

- **Scoring rules differed.** For vector RAG and long context, a number-matcher graded every question and its verdict was final. It misreads descriptive answers: “growth was *flat* in FY2023” was marked wrong because it read 2023 as the answer. I switched everything to one rule: the matcher on calculation questions only, the judge for the rest. Long context went from 70.0% to 81.3%.
- **Prompts differed.** Vectorless and the hybrid used a structured prompt that asks the model to show its calculation, while the others used a short one. I re-ran vector RAG and long context with the shared prompt, and re-timed them. Vector RAG rose to 70.7%; long context settled at 78.7%.

Lesson: evaluation choices moved one architecture's accuracy by up to 17 points, more than most differences between architectures. A comparison only means something when prompt and scoring are identical.

5. Results

Architecture	Accuracy	Correct / Incorrect / Refused	Cost per 1k queries	Median latency	p95 latency	One-time indexing
Vectorless RAG	79.3%	119 / 28 / 3	\$12.80	3.6 s	4.2 s	~\$15
Long context	78.7%	118 / 27 / 5	\$19.96	30.3 s	134.6 s	none
PageIndex hybrid	74.0%	111 / 25 / 14	\$4.54	4.6 s	28.8 s	~\$15 (reused)
Vector RAG	70.7%	106 / 22 / 22	\$1.91	5.1 s	7.8 s	\$0 (local GPU)



Pareto views (log x-axes). Filled points are on that view's frontier; hollow points are dominated. Latency: median, line to p95.

- **Vectorless RAG is the strongest option.** It's as accurate as long context (79.3% vs 78.7%, $p = 1.0$), about 8× faster and 36% cheaper per query, so **long context is dominated** and drops off the frontier.
- **Statistical significance (paired McNemar tests):** only vectorless vs. vector RAG is clearly significant ($p = 0.035$). Vector RAG vs. long context is borderline ($p = 0.05$); all other gaps are within what 150 questions can resolve.
- **Long context is slow because of the PDF input**, most likely: one filing took about 2.3 minutes per answer. Vectorless reads ~33k tokens of plain text in about a second.

6. What I saw in each architecture

Vector RAG

- Retrieval works: a gold page is in the reranked top 10 for 88% of questions. But the **selection agent keeps only 1.6 of 10 chunks**, and it discarded every gold-page chunk in 28 of 132 questions. The model then reads just ~800 tokens, the least of any architecture.
- Removing the selection agent leaves accuracy unchanged (70.0% vs 70.7%), but changes behaviour (Section 7).

Vectorless RAG

- Navigation is accurate (94% hit rate), and 27 of its 31 misses happened *after* navigating to the right section, so errors are reading or reasoning, not finding.
- It succeeds by reading a lot: a median of 36 pages (about 40% of the document), which is where its cost comes from. The fragile part is building the trees, not navigating them.

PageIndex hybrid

- Narrowing first doesn't raise recall (86% vs 88%), but makes retrieval the most *precise* of all (14.6% of retrieved pages are evidence). Accuracy of 74.0% is not significantly different from the control.
- Follow-up: using PageIndex as a *filter* after reranking cut tokens by 31% but lowered accuracy to 66.0% (refusals rose from 14 to 23). PageIndex works better as a guide before retrieval than as a filter after it.

Long context

- Needs no preprocessing, and the largest filing (307k tokens) still fit the window. Its misses are mostly wrong answers (27) rather than refusals (5). It is the slowest and most expensive architecture per query.

7. Digging deeper

Refusals depend on the question type

Failure to answer	Calculation	Domain-relevant	Novel-generated
Vector RAG	8%	16%	20%
PageIndex hybrid	6%	6%	16%
Vectorless RAG	0%	0%	6%
Long context	0%	2%	8%

The chunk-based pipelines refuse far more often on descriptive questions, where the answer is spread across a section's narrative. By reasoning type, vector RAG is actually the **best** at simple information extraction (91%), but far behind on numerical reasoning (72% vs 89% for vectorless and long context), because calculations combine figures from several places.

Recall rewards retrieving more, so I also measured focus

Retrieval	Recall	MRR@10	Pages retrieved	Precision
Vector RAG: rerank top 10	86.1%	0.608	9	12.0%
PageIndex hybrid: rerank top 10	84.4%	0.608	8	14.6%
Vectorless: navigated sections	93.3%	n/a	36	7.2%

Vectorless has the highest recall because it retrieves the most. Chunk retrieval is twice as focused, yet the unfocused retrieval gives the most accurate answers: for FinanceBench, missing surrounding context costs more than including irrelevant pages.

Why novel-generated questions are hard for the chunk-based pipelines

I checked whether the gold evidence text actually reached the model, using ROUGE-2 recall (the share of the evidence's word pairs found in the chunks). On novel questions:

- **Vector RAG's wrong answers had seen only 17% of the evidence** (93% when correct). It answered from the wrong part of a table that its 1–2 chunks had cut apart.
- **The hybrid's wrong answers had seen 99% of the evidence.** It had the text and misread it: a different definition, sign or scope than the gold answer.

Novel questions ask for derived analyst concepts (free-cash-flow conversion, "real" growth), so they need a whole table *and* the right interpretation. Some also allow more than one reasonable reading, which hurts every architecture (60–70% correct).

What the selection agent really does

Without the selection agent, vector RAG sees the evidence far more often (ROUGE-2 on wrong novel answers: 17% → 100%), yet accuracy stays the same. Gains and losses cancel out: 9 questions got fixed, but 9 correct answers went wrong (extra chunks bring similar-looking figures from other years or segments) and 6 honest refusals became wrong answers. With selection, the model is right **83%** of the times it answers, against **77%** without. The same accuracy hides a real difference in reliability: **it refuses instead of guessing.**

8. Cost over time: when does indexing pay off?

Total cost	100 queries	1,000 queries	10,000 queries	100,000 queries
Vector RAG	\$0.19	\$1.91	\$19.07	\$190.66
PageIndex hybrid	\$15.11	\$19.20	\$60.10	\$469.12
Vectorless RAG	\$15.93	\$27.45	\$142.62	\$1,294.37
Long context	\$2.00	\$19.96	\$199.58	\$1,995.84

Vectorless's ~\$15 of tree building pays for itself after about **2,000 queries** (roughly 24 per filing), and the hybrid's after about 950. Vector RAG is the cheapest at any volume. For any real use where a filing is queried more than a handful of times, per-query cost matters far more than indexing.

9. Conclusions

- **RQ1 (accuracy):** vectorless 79.3% $\approx$ long context 78.7% > hybrid 74.0% > vector RAG 70.7%.
- **RQ2 (retrieval, latency, cost):** retrieval finds a gold page 86–94% of the time everywhere; vectorless, hybrid and vector RAG answer in ~4–5 s, long context in 30 s; cost spans more than 10 $\times$ (\$1.91–\$19.96 per 1k queries).
- **RQ3 (dominance):** no single winner, but **vectorless dominates long context**. Vectorless for accuracy and speed, vector RAG for cost, the hybrid as a middle ground.
- **RQ4 (frontier):** vectorless $\rightarrow$ hybrid $\rightarrow$ vector RAG, trading accuracy for cost.

The finding that ties it together: accuracy follows how much relevant context reaches the model, up to about section level: ~0.8k tokens $\rightarrow$ 70.7%, 4.4k $\rightarrow$ 74.0%, 33k $\rightarrow$ 79.3%. The whole filing (70k tokens, 78.7%) adds nothing and is 8 $\times$ slower. Retrieval is not the bottleneck; errors split into **not seeing the evidence** (chunk-based, fixable with more context) and **misreading it** (a limit of the model and of question ambiguity).

10. Limitations and next steps

- One generator only; DeepSeek V4 is not yet run (its API takes no PDF input, so long context needs a text-only path).
- Each architecture was generated once; 3 of 26 questions with identical input flipped between runs, so gaps of a few questions are within noise. Repeated runs would tighten this.
- Latency samples are small (15–30 runs; vectorless only 5 questions) and some pipelines were timed on different days, so p95 is indicative only.
- Jina rerank is unpriced; PageIndex indexing cost is partly estimated; vectorless token counts are local estimates.
- The judge grades only the final answer field, not the reasoning, and its verdicts still need the planned manual 10–15% spot-check.

Full analysis with live code: [analysis/findings_walkthrough.ipynb](#) (GitHub: shaliqsv/financebench-rag-thesis).